

C++ Interview Test Bank

Constructors, Destructors, Copy/Move, const/static/constexpr, Virtual/Polymorphism, Casts, References

Այս ֆայլը միայն այն թեմաներից է, որ արդեն ուսումնասիրել ենք: Երկու մաս.

Մաս A — 50 multiple-choice թեստ (4 տարբերակ, մեկը ճիշտ), կոդի օրինակներով՝ ինչ կտաի, error է թե ոչ, ինչ error:

Մաս B — 50 բաց (open) interview հարց՝ առանց տարբերակների:

Բոլոր պատասխանները **միայն վերջի էջերում** են (Answer Key A և Answer Key B): Առաջ նայելուց առաջ փորձիր ինքդ պատասխանել:

ՄԱՍ A — Multiple Choice (50)

Q1

```
class A { public: A() { cout << "A"; } };  
int main() { A a; }
```

Ի՞նչ կտպի: - (a) ոչինչ - (b) A - (c) compile error - (d) AA

Q2

Constructor-ի return type-ը ի՞նչ է: - (a) void - (b) int - (c) նույն class-ի type-ը - (d) չունի return type

Q3

```
class A { public: void A() {} };
```

Ի՞նչ է սա: - (a) նորմալ constructor - (b) compile error (constructor-ը return type չունի) - (c) destructor - (d) copy constructor

Q4

```
class A {  
public:  
    A() { cout << "default "; }  
    A(int x) { cout << "param "; }  
};  
int main() { A a; A b(5); }
```

Ի՞նչ կտպի: - (a) default param - (b) param default - (c) default default - (d) param param

Q5

Ո՞ր constructor-ն է կանչվում `A b = a;` (a-ն արդեն A object է): - (a) default constructor - (b) parameterized constructor - (c) copy constructor - (d) move constructor

Q6

```
class A { A() {} };  
int main() { A a; }
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (constructor-ը private է) - (c) runtime error - (d) տպում է A

Q7

Ի՞նչ է Initialization List-ը `A(int n) : x(n) {}` -ում: - (a) x-ը assign է անում մարմնում - (b) x-ը initialize է անում մարմնից առաջ - (c) նոր function է - (d) compile error

Q8

Constructor-ը կարող է virtual լինել: - (a) այո - (b) ոչ, քանի որ object-ը դեռ գոյություն չունի - (c) միայն abstract class-ում - (d) միայն C++11-ից

Q9

Destructor-ի syntax-ը ճիշդ է: - (a) `!ClassName()` - (b) `~ClassName()` - (c) `delete ClassName()` - (d) `ClassName::~~()`

Q10

```
{ Test t; } cout << "end";
```

Test-ը ունի destructor, որ տպում է "D": Ի՞նչ հերթականությամբ: - (a) end D - (b) D end - (c) միայն end - (d) միայն D

Q11

Heap object-ի destructor-ը երբ է կանչվում:

```
Test* p = new Test();
```

- (a) scope-ից դուրս ավտոմատ
- (b) երբ `delete p;` անես
- (c) երբեք
- (d) program start-ին

Q12

Destructor-ը կարող է parameter ունենալ: - (a) այո - (b) ոչ - (c) միայն int - (d) միայն default

Q13

Քանի՞ destructor կարող է ունենալ class-ը: - (a) անսահման - (b) 2 - (c) 1 - (d) 0

Q14

```
Test a; Test b;
```

LIFO-ի դեպքում որ destructor-ն է առաջինը կանչվում: - (a) a-ինը - (b) b-ինը - (c) միաժամանակ - (d) ոչ մեկը

Q15

```
class Base { public: ~Base(){cout<<"B";} };  
class Derived: public Base { public: ~Derived(){cout<<"D";} };  
Base* p = new Derived();  
delete p;
```

Ի՞նչ կտպի (destructor-ը virtual Չէ): - (a) DB - (b) BD - (c) B - (d) D

Q16

Նույն Q15-ը, բայց `virtual ~Base()`: Ի՞նչ կտպի: - (a) B - (b) D - (c) DB - (d) BD

Q17

Ինչո՞ւ է virtual destructor-ը կարևոր: - (a) արագացնում է կոդը - (b) որ base pointer-ով delete-ի ժամանակ derived destructor-ն էլ կանչվի - (c) memory խնայում է - (d) պարտադիր է ամեն class-ում

Q18

```
class Test { int* data; public:  
    Test(){ data=new int(5); }  
    ~Test(){ delete data; }  
};  
Test a; Test b = a;
```

Ի՞նչ խնդիր կա (default copy): - (a) ոչ մի - (b) shallow copy -> double delete - (c) compile error - (d) memory քիչ է

Q19

Deep copy-ի ճիշտ copy constructor-ը դրո՞ւմ է: - (a) `Test(const Test& o){ data=o.data; }` - (b) `Test(const Test& o){ data=new int(*o.data); }` - (c) `Test(Test o){ data=o.data; }` - (d) `Test(){ data=new int; }`

Q20

Ի՞նչո՞ւ copy constructor-ը `const Test&` է ընդունում, ոչ `Test`: - (a) գեղեցիկ է - (b) որ copy չլինի և infinite recursion-ից խուսափի - (c) compile error է լինում այլապես միշտ - (d) տարբերություն չկա

Q21

```
void f(std::string& s){}
f("Arman");
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (non-const ref-ը temporary չի ընդունում) - (c) runtime error - (d) տպում է Arman

Q22

```
void f(const std::string& s){}
f("Arman");
```

Ի՞նչ կլինի: - (a) OK - (b) compile error - (c) runtime error - (d) memory leak

Q23

```
void f(std::string&& s){}
f("Arman");
```

Ի՞նչ կլինի: - (a) compile error - (b) OK (rvalue reference-ը temporary-ն ընդունում է) - (c) runtime error - (d) infinite loop

Q24

`std::move(x)` իրականում ի՞նչ է անում: - (a) տեղափոխում է x-ի տվյալները - (b) ջնջում է x-ը - (c) cast է անում rvalue-ի (`static_cast<T&&>`) - (d) copy է անում

Q25

```
int&& r = 5;
```

`r`-ը հետո օգտագործելիս ի՞նչ է: - (a) rvalue - (b) lvalue (անուն ունի) - (c) pointer - (d) const

Q26

```
int* a = new int;  
cout << *a;
```

Ի՞նչ կտպի: - (a) 0 - (b) undefined (աղբ արժեք) - (c) compile error - (d) nullptr

Q27

```
int* b = new int();  
cout << *b;
```

Ի՞նչ կտպի: - (a) undefined - (b) 0 - (c) compile error - (d) 1

Q28

`std::string_view`-ը ի՞նչ է պահում: - (a) string-ի ամբողջ copy - (b) pointer + length - (c) միայն length - (d) heap allocation

Q29

SSO (Small String Optimization) ի՞նչ է անում: - (a) փոքր string-ը պահում է object-ի ներսում, heap չի գնում - (b) string-ը compress է անում - (c) միշտ heap է օգտագործում - (d) string-ը ջնջում է

Q30

```
const int x = 10;  
x = 20;
```

Ի՞նչ կլինի: - (a) OK, x=20 - (b) compile error (x-ը const է) - (c) runtime error - (d) x=10 մնում է լուռ

Q31

```
const int* p;
```

Ի՞նչն է const: - (a) pointer-ը - (b) տվյալը (որին ցույց է տալիս) - (c) երկուսն էլ - (d) ոչ մեկը

Q32

```
int* const p = &x;
```

Ի՞նչն է const: - (a) տվյալը - (b) pointer-ը - (c) երկուսն էլ - (d) ոչ մեկը

Q33

```
void foo(){ static int x=0; x++; cout<<x; }  
foo(); foo(); foo();
```

Ի՞նչ կտպի: - (a) 111 - (b) 123 - (c) 000 - (d) 333

Q34

Static class member-ը ղում է պատկանում: - (a) յուրաքանչյուր object-ին առանձին - (b) class-ին (մեկ instance բոլորի համար) - (c) main function-ին - (d) ոչ մեկին

Q35

```
class A { int age; public: static void f(){ cout<<age; } };
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (static function-ը non-static member չի տեսնում) - (c) տպում է 0 - (d) runtime error

Q36

`const` vs `constexpr` — ի՞նչն է ճիշտ: - (a) նույն բանն են - (b) `constexpr` = compile-time constant, `const` = immutable object - (c) `const` = compile-time միայն - (d) `constexpr`-ը runtime-ում է

Q37

```
Animal* p = new Dog();  
p->speak();
```

`speak()` virtual Չէ: Base-ը տպում է "Animal", Dog-ը "Dog": Ի՞նչ կտպի: - (a) Dog - (b) Animal - (c) compile error - (d) AnimalDog

Q38

Նույնը, բայց `speak()` virtual է: Ի՞նչ կտպի: - (a) Animal - (b) Dog - (c) compile error - (d) ոչինչ

Q39

`virtual void f() = 0;` ինչ է: - (a) սովորական virtual - (b) pure virtual (class-ը abstract է դառնում) - (c) static function - (d) compile error

Q40

```
class Shape { public: virtual double area()=0; };  
Shape s;
```

Ի՞նչ կլինի: - (a) OK - (b) compile error (abstract class-ից object չի ստեղծվում) - (c) runtime error - (d) area()=0

Q41

vtable-ը ռմբ է պատկանում: - (a) object-ին - (b) class-ին (մեկ հատ per polymorphic class) - (c) function-ին - (d) vptr-ին

Q42

vptr-ը որտեղ է պահվում: - (a) class-ում - (b) object-ի ներսում - (c) stack-ում առանձին - (d) vtable-ում

Q43

```
class Base { public: virtual void f(int){} };  
class Derived: public Base { public: void f(double){} };
```

Derived-ի `f(double)` -ը ինչ է Base-ի նկատմամբ: - (a) override - (b) overload - (c) ոչ override, ոչ overload (նոր function, name hiding) - (d) compile error

Q44

`override` keyword-ը ինչի համար է: - (a) արագացնում է - (b) compiler-ը բռնում է սխալ signature-ը - (c) պարտադիր է virtual-ի հետ - (d) memory խնայում է

Q45

```
class A {};  
A* a = new B();  
B* b = dynamic_cast<B*>(a);  
// ոչ polymorphic  
// B: public A
```

Ի՞նչ կլինի: - (a) OK - (b) compile error ('A' is not polymorphic) - (c) nullptr - (d) valid pointer

Q46

```
Base* p = new Cat();           // Cat: public Animal, polymorphic
Dog* d = dynamic_cast<Dog*>(p);
```

d -ն ի՞նչ է: - (a) valid Dog* - (b) nullptr - (c) compile error - (d) undefined

Q47

Reference-ի dynamic_cast ձևափոխվելիս ի՞նչ է լինում: - (a) nullptr - (b) throw std::bad_cast - (c) compile error - (d) 0

Q48

```
const int y = 5;
int* q = const_cast<int*>(&y);
*q = 10;
```

Ի՞նչ է սա: - (a) OK, y=10 - (b) compile error - (c) undefined behavior (իսկապես const object-ի փոփոխում) - (d) y=5 երաշխավորված

Q49

C-style **(T)x** cast-ը ի՞նչ հերթականությամբ է compiler-ը փորձում: - (a) միայն static_cast - (b) const -> static -> static+const -> reinterpret -> reinterpret+const - (c) միայն reinterpret_cast - (d) dynamic_cast առաջինը

Q50

```
Base* b = new Base();
Derived* d = static_cast<Derived*>(b); // downcast, b-ն իրականում Base է
```

Ի՞նչ է սա: - (a) compile error - (b) nullptr - (c) կոմպիլվում է, բայց undefined behavior (static_cast չի ստուգում) - (d) valid pointer

ՄԱՍ B — Open Questions (50)

Սրանք բաց հարցեր են (առանց տարբերակների): Փորձիր բարձրաձայն, ամբողջական պատասխանել, ասես interview-ում ես: Մոդելային պատասխանները՝ Answer Key B-ում:

1. Ի՞նչ է constructor-ը և ինչու է պետք:
2. Constructor-ը return type ունի, և ինչու:
3. Ի՞նչ է default constructor-ը:
4. Ի՞նչ է parameterized constructor-ը:
5. Ի՞նչ է constructor overloading-ը:
6. Ի՞նչ է initialization list-ը և ինչու է assignment-ից լավ:
7. Constructor-ը կարող է virtual լինել: Ինչու:
8. Constructor-ը կարող է private լինել: Որտեղ է դա օգտագործվում:
9. Բացատրիր Singleton pattern-ը և ինչու է constructor-ը private:
10. Ի՞նչ է destructor-ը և երբ է կանչվում:
11. Stack և heap object-ների destructor-ը երբ է կանչվում:
12. Ինչու է destructor-ը պետք (resource management):
13. Destructor-ը parameter կամ return type ունի:
14. Կարող է class-ը մի քանի destructor ունենալ:
15. Ի՞նչ է destructor call order-ը (LIFO):
16. Ի՞նչ է virtual destructor-ը և ինչու է կարևոր:
17. Ի՞նչ է լինում, եթե base destructor-ը virtual չէ և delete անես base pointer-ով:
18. Ի՞նչ է copy constructor-ը:
19. Երբ է copy constructor-ը կանչվում (3 դեպք):
20. Ի՞նչ է shallow copy-ն:
21. Ի՞նչ է deep copy-ն:
22. Ինչո՞ւ copy constructor-ը `const&` -ով է ընդունում argument-ը:
23. Ի՞նչ է double delete-ը և ինչպես է առաջանում:
24. Ի՞նչ է move constructor-ը և ինչու է copy-ից արագ:
25. Ի՞նչ է rvalue reference-ը (`&&`):
26. `std::move` -ը իրականում ի՞նչ է անում:
27. Բացատրիր lvalue և rvalue-ի տարբերությունը:
28. Ինչու է `int&& r = 5;` -ից հետո `r` -ը lvalue:
29. Ի՞նչ է Rule of Zero-ն:
30. `new int` vs `new int()` — ի՞նչ տարբերություն:
31. Ի՞նչ է temporary object-ը:
32. Ի՞նչ է `std::string_view` -ը և ինչու է օգտիմալ:
33. Ի՞նչ են RVO և NRVO-ն:
34. Ի՞նչ է SSO-ն:

35. Ի՞նչ է `const`-ը և ինչ է երաշխավորում:
 36. `const int*` vs `int* const` — բացատրիր:
 37. Ի՞նչ է const member function-ը:
 38. Ինչու `const string&` parameter — 2 պատճառ:
 39. Ի՞նչ է static local variable-ը և երբ է ստեղծվում/մահանում:
 40. Static class member-ը ո՞ւմ է պատկանում և ինչպես ես define անում:
 41. Ինչու static member function-ը չի կարող non-static member օգտագործել:
 42. `const` vs `constexpr` — տարբերությունը:
 43. Ի՞նչ է virtual function-ը և runtime polymorphism-ը:
 44. Static binding vs dynamic binding — բացատրիր:
 45. Override vs overload — մանրամասն տարբերությունը:
 46. Ի՞նչ է vtable-ը և vptr-ը, դր level-ում են:
 47. Քայլ առ քայլ ի՞նչ է կատարվում virtual call-ի ժամանակ:
 48. Ի՞նչ է pure virtual function-ը և abstract class-ը:
 49. Բացատրիր 4 C++ cast-երը և երբ որն օգտագործել:
 50. static_cast vs dynamic_cast downcast-ի ժամանակ — տարբերությունը:
-

ANSWER KEY A (Multiple Choice)

1-b · 2-d · 3-b · 4-a · 5-c · 6-b · 7-b · 8-b · 9-b · 10-b 11-b · 12-b · 13-c · 14-b · 15-c · 16-c · 17-b · 18-b · 19-b · 20-b 21-b · 22-a · 23-b · 24-c · 25-b · 26-b · 27-b · 28-b · 29-a · 30-b 31-b · 32-b · 33-b · 34-b · 35-b · 36-b · 37-b · 38-b · 39-b · 40-b 41-b · 42-b · 43-c · 44-b · 45-b · 46-b · 47-b · 48-c · 49-b · 50-c

Կարճ բացատրություններ (A)

Q1 `A a;` կանչում է constructor-ը -> "A": **Q3** `void A()` return type ունի -> constructor չէ, բայց նաև method-ը class-ի անունով հակասական է; գործնականում compile error է constructor-ի կանոնի խախտմամբ: **Q5** `A b = a;` copy initialization -> copy constructor: **Q10** Stack object-ի destructor-ը scope-ի վերջում, բայց "end"-ից հետո? Ոչ — `t` -ն ապրում է մինչ block-ի `}`; "end" տողը block-ից դուրս է, ուստի D-ն տպվում է block-ը փակվելիս, հետո... իրականում այստեղ `{Test t;}` block-ն ավարտվում է cout-ից առաջ, ուստի **D end**: **Q15** virtual չկա -> միայն Base destructor -> "B": **Q16** virtual կա -> "DB" (Derived հետո Base): **Q18** default copy -> shallow -> երկուսը նույն pointer -> double delete: **Q21** non-const lvalue ref-ը temporary չի ընդունում -> compile error: **Q24** `std::move` = `static_cast<T&&>`, ոչինչ չի տեղափոխում: **Q25** named rvalue reference -> lvalue: **Q26** `new int` uninitialized -> undefined/աղբ: **Q30** const object-ի reassignment -> compile error: **Q35** static function-ը `this` չունի -> non-static `age` չի տեսնում -> compile error: **Q37** virtual չկա -> static binding -> "Animal": **Q40** abstract class -> object չի ստեղծվում -> compile error: **Q43** signature տարբեր -> ոչ override, ոչ overload; name hiding: **Q45** `A` polymorphic չէ -> `dynamic_cast` compile error: **Q46** իրական object Cat է, ոչ Dog -> nullptr: **Q48** իսկապես const object-ի փոփոխում -> UB: **Q50** `static_cast` downcast-ը չի ստուգում -> UB:

ANSWER KEY B (Open Questions)

1. Constructor-ը class-ի հատուկ member function է, որ ավտոմատ կանչվում է object ստեղծելիս; նպատակը՝ object-ի data member-ները initialize անել և պատրաստել օգտագործման: Անունը class-ի անունն է, return type չունի:
2. Ոչ, return type չունի (նույնիսկ void ոչ): Քանի որ նրա նպատակը object-ի կառուցումն է, ոչ արժեք վերադարձնելը; լեզուն այդպես է սահմանված:
3. Parameter չունեցող constructor; կանչվում է `A a;` դեպքում; եթե ոչ մի constructor չգրես, compiler-ը գեներացնում է implicit default constructor:
4. Constructor, որ ընդունում է parameter(ներ) և դրանցով initialize է անում member-ները, օր.՝
`Person(string n): name(n){}` :
5. Նույն class-ում մի քանի constructor՝ տարբեր parameter list-երով; compiler-ը ընտրում է ըստ argument-ների (compile-time):
6. Init list-ը member-ները initialize է անում մարմնից առաջ՝ ուղղակի construction-ով, ոչ թե default-construct + assign; ավելի արդյունավետ է, և `const` /reference member-ների ու base class-ի համար պարտադիր է:
7. Ոչ: Virtual dispatch-ը հիմնվում է vptr-ի վրա, որը դեռ set չէ object-ի կառուցման պահին; բացի այդ object-ը դեռ գոյություն չունի, ուստի «որ տիպի» constructor-ը կանչել իմաստ չունի:
8. Այո: Օգտագործվում է, երբ ուզում ես կառավարել object-ի ստեղծումը՝ Singleton, factory method, կամ արգելել ուղղակի instantiation:
9. Singleton-ը երաշխավորում է class-ի միայն մեկ instance; constructor-ը private է, որ դրսից object չստեղծվի, և `static getInstance()` -ը վերադարձնում է միակ static instance-ը (`static T instance;`):
10. Destructor-ը հատուկ member function է (`~Class()`), որ ավտոմատ կանչվում է object-ի ոչնչացման ժամանակ; նպատակը՝ resource cleanup (memory, file, socket):
11. Stack object -> scope-ից դուրս գալիս ավտոմատ; heap object (`new`) -> միայն `delete` -ի ժամանակ:
12. Որ object-ի բռնած resource-ները ազատվեն (RAII): heap memory, file handle, DB connection, lock; հակառակ դեպքում leak կամ resource exhaustion:
13. Ոչ մեկը. destructor-ը parameter չունի (ուստի overloading հնարավոր չէ) և return type չունի:
14. Ոչ, միայն մեկ destructor:
15. LIFO — վերջին կառուցված object-ն առաջինը ոչնչանում; `Test a; Test b;` -> ~b հետո ~a:

16. Base destructor-ը virtual դարձնելը երաշխավորում է, որ base pointer-ով delete անելիս derived destructor-ն էլ կանչվի; կարևոր է polymorphic base class-ների համար՝ leak-ից խուսափելու:

17. Կանչվում է միայն base destructor-ը (static binding), derived-ի cleanup-ը չի կատարվում -> resource/memory leak, undefined behavior:

18. Special constructor, որ նոր object է ստեղծում գոյություն ունեցող object-ի հիման վրա.

`A(const A& other) :`

19. (1) `A b = a;` / `A b(a);` initialization; (2) function-ին by value փոխանցում; (3) function-ից by value return (հաճախ optimize է լինում):

20. Shallow copy — պատճենում է միայն pointer-ի արժեքը (հասցեն); երկու object նույն memory-ն են ցույց տալիս -> shared state, double delete վտանգ:

21. Deep copy — նոր memory է allocate անում և պատճենում իրական բովանդակությունը; object-ները անկախ են:

22. Որ (1) copy չլինի (reference), և հատկապես որ (2) infinite recursion-ից խուսափի — by value parameter-ը ինքը copy constructor կպահանջեր; `const` -ը երաշխավորում է original-ը չփոխվի:

23. Նույն pointer-ը երկու անգամ delete անելը (shallow copy-ից հետո երկու object նույն memory-ն են ազատում) -> undefined behavior, crash/heap corruption:

24. Move constructor-ը resource-ի ownership է փոխանցում (pointer «գողանում»), նոր allocation/copy չի անում; դրա համար մեծ object-ների համար շատ ավելի արագ է քան deep copy:

25. rvalue reference (`T&&`) bind է լինում rvalue-ների (temporary-ների) հետ; move semantics-ի հիմքն է. թույլ է տալիս «գողանալ» temporary-ի resource-ները:

26. Ոչինչ չի տեղափոխում. պարզապես `static_cast<T&&>` է անում, որ object-ը treat լինի որպես rvalue և move constructor/assignment-ը ընտրվի:

27. lvalue — ունի անուն ու հասցե, կարող ես `&` -ով վերցնել (օր. `x` , `arr[0]`); rvalue — temporary արժեք, address չունի (օր. `5` , `x+1` , `foo()`):

28. Քանի որ ցանկացած named variable lvalue է, նույնիսկ եթե տիպը rvalue reference է; `r` -ն անուն ունի, ուստի օգտագործելիս lvalue է: Move անելու համար պետք է `std::move(r)` :

29. Rule of Zero — եթե օգտագործես RAII տիպեր (`std::string` , `std::vector` , `std::unique_ptr`), ընդհանրապես ձեռքով copy/move/destructor գրելու կարիք չկա; compiler-ի default-երը ճիշտ են աշխատում:

30. `new int` — uninitialized (աղբ արժեք, UB կարդալիս); `new int()` — value-initialized 0-ով; `new int{}` (C++11) նույնպես 0:

31. Object, որ compiler-ը ստեղծում է կարճ ժամանակով (օր. `std::string("x")` , `a+b`); սովորաբար expression-ի վերջում ոչնչանում է (եթե const ref-ը չերկարացնի lifetime-ը):

32. Non-owning view, որ պահում է միայն pointer + length; ընթերցման համար string-ի copy/allocation չի անում, ուստի function parameter-ի համար օպտիմալ է, երբ միայն կարդում ես: Զգույշ dangling-ից:

33. RVO/NRVO — compiler optimization-ներ, որ վերացնում են return value-ի copy/move-ը՝ object-ն ուղղակի կառուցելով վերջնական տեղում; C++17-ում RVO-ն (temporary-ի դեպքում) երաշխավորված է:

34. Small String Optimization — փոքր string-երը պահվում են հենց `std::string` object-ի ներսում (buffer), առանց heap allocation; ուստի կարճ string-երը շատ արագ են:

35. `const` -ը երաշխավորում է, որ object-ը initialization-ից հետո չի փոփոխվի; օգնում է անվտանգության և optimization-ի; lifetime-ը չի փոխում:

36. `const int* p` — տվյալն է `const` (`*p` -ն չես կարող փոխել, `p` -ն կարող ես); `int* const p` — pointer-ն է `const` (`p` -ն չես կարող փոխել, `*p` -ն կարող ես): Կարդա աջից ձախ:

37. `T f() const;` — երաշխավորում է, որ function-ը չի փոփոխի object-ի (non-mutable) member-ները; `this` -ը դառնում է `const`; `const` object-ների վրա միայն `const` method կարելի է կանչել:

38. (1) Copy չի լինում (reference, մեծ object-ի համար էժան); (2) function-ը չի կարող փոխել original-ը (`const`): Բռնու՝ ընդունում է և lvalue, և temporary:

39. Variable, որ ստեղծվում է առաջին անգամ function կանչվելիս և ապրում մինչև program end՝ պահպանելով արժեքը կանչերի միջև; initialization-ը կատարվում է մեկ անգամ (thread-safe C++11-ից):

40. Class-ին (ոչ object-ին) — մեկ shared instance բոլոր object-ների համար; define-ը class-ից դուրս՝ `int A::count = 0;`; հասանելի է `A::count` -ով:

41. Քանի որ static function-ը object-ի հետ կապ չունի, չունի `this` pointer; non-static member-ը կոնկրետ object է պահանջում, ուստի static function-ից անհասանելի է:

42. `const` — immutable object (կարող է runtime-ում initialize լինել); `constexpr` — compile-time constant/function (արժեքը հայտնի է compile time-ում): Ամեն `constexpr const` է, բայց ոչ հակառակը:

43. Virtual function-ը թույլ է տալիս call-ը resolve լինել runtime-ում ըստ object-ի իրական տիպի (ոչ pointer-ի static տիպի); սա runtime polymorphism-ն է՝ նույն interface, տարբեր behavior:

44. Static (early) binding — call-ը որոշվում է compile-time-ում (non-virtual function); dynamic (late) binding — runtime-ում vptr/vtable-ով (virtual function):

45. Override — derived class-ը վերասահմանում է base-ի virtual function-ը նույն signature-ով (runtime, inheritance պետք է); Overload — նույն անունով function-ներ տարբեր parameter-ներով (compile-time, inheritance պետք չէ): Signature-ի տարբերությունը override-ը վերածում է name hiding-ի:

46. vtable — per-class աղյուսակ virtual function-ների հասցեներով (class level, մեկ հատ); vptr — per-object hidden pointer, որ ցույց է տալիս class-ի vtable-ին (object level):

47. (1) object-ից կարդում vptr; (2) vptr-ով մտնում vtable; (3) գտնում function-ի slot-ի հասցեն; (4) jump/call այդ հասցեին -> իրական (derived) implementation:

48. Pure virtual function (= 0) — implementation չունեցող virtual, որ derived-ը պարտավոր է override անել; class-ը, որ ունի գոնե մեկ pure virtual, abstract է -> object չի ստեղծվում, բայց pointer/reference կարելի է:

49. `static_cast` — ստուգված compile-time փոխակերպումներ (թվային, upcast, void*);

`dynamic_cast` — runtime-ստուգված downcast polymorphic տիպերի; `const_cast` — const/volatile ավելացնել/հեռացնել; `reinterpret_cast` — bit-level, ամենավտանգավոր: Խուսափիր C-style cast-ից:

50. static_cast downcast-ը կոմպիլվում է բայց runtime-ում չի ստուգում -> սխալ տիպի դեպքում UB; dynamic_cast-ը ստուգում է RTTI-ով -> ձախողման դեպքում nullptr (pointer) կամ bad_cast (reference)՝ ապահով: